

```
timescale 1ns / 1ps
`default_nettype none
`include "IF.v"
`include "ID.v"
`include "EX.v"
`include "MEM.v"
`include "WB.v"
`include "hazard_unit.v"
```

```
module core(
    input wire clk,
    input wire rst_n,
    input wire [31:0] i_i_data,
    input wire [31:0] i_d_data,
    input wire i_i_ack,
    input wire i_d_ack,
    output wire o_i_stb,
    output wire o_d_stb,
    output wire o_d_wr_en,
    output wire [31:0] o_d_write_data,
    output wire [31:0] o_i_addr,
    output wire [31:0] o_d_addr,
    output wire [31:0] if_instr, if_Pc,
    output wire [31:0] id_rs1_data, id_rs2_data, id_imm_data, id_pc,
    output wire [2:0] id_func3,
    output wire [4:0] id_rd,
    output wire id_regSrc,
    output wire id_sel1,
    output wire id_sel2,
    output wire id_load,
    output wire id_branch,
    output wire id_jal,
    output wire id_jalr,
    output wire id_memSrc,
    output wire [1:0] id_resultSrc,
    output wire [3:0] id_alu_ctrl,
    output wire [31:0] ex_pc, ex_imm_data, ex_data_store,
    output wire [2:0] ex_func3,
    output wire [4:0] ex_rd,
    output wire ex_load,
    output wire [31:0] result_to_if,
    output wire ex_boj,
    output wire ex_jalr,
    output wire ex_regsrc,
    output wire ex_memsrc,
    output wire [31:0] ex_result,
```

```

output wire [1:0] ex_resultsrc,
output wire [31:0] fwdwire1, fwdwire2,
output wire m_regSrc,
output wire [4:0] m_rd,
output wire [1:0] m_resultsrc,
output wire [31:0] m_result,
output wire [31:0] m_data,
output wire [31:0] m_pc4,
output wire w_regsrc,
output wire [4:0] w_rd,
output wire [31:0] w_data,
output wire o_load_use_hazard,
output wire [4:0] id_amo_op, ex_amo_op, m_amo_op
);

```

```

wire [1:0] mux1_sel, mux2_sel;
wire [4:0] is_Rs1, is_Rs2;
wire [4:0] is_Rs1E, is_Rs2E;

```

```

// IF Stage - Instruction Fetch

```

```

IF if_module (
    .clk(clk),
    .rst_n(rst_n),
    .stall(o_load_use_hazard),
    .i_Inst(i_i_data),
    .i_Imem_ack(i_i_ack),
    .o_Imem_stb(o_i_stb),
    .o_Iaddr(o_i_addr),
    .i_Imm(result_to_if),
    .i_Result(ex_result),
    .i_Boj(ex_boj),
    .i_Jalr(ex_jalr),
    .o_InstrD(if_instr),
    .o_PcD(if_Pc)
);

```

```

// ID Stage - Instruction Decode

```

```

ID id_module (
    .clk(clk),
    .rst_n(rst_n),
    .i_Instr(if_instr),
    .i_Pc(if_Pc),
    .i_write_data(w_data),
    .i_wr(w_regsrc),
    .i_Rd(w_rd),
    .i_Boj(ex_boj),

```

```

        .flush(o_load_use_hazard),
        .o_Func3E(id_func3),
        .o_Rd1E(id_rs1_data),
        .o_Rd2E(id_rs2_data),
        .o_ImmE(id_imm_data),
        .o_PcE(id_pc),
        .o_RdE(id_rd),
        .o_RegSrcE(id_regSrc),
        .o_Sel1E(id_sel1),
        .o_Sel2E(id_sel2),
        .o_ALUCtrlE(id_alu_ctrl),
        .o_LoadE(id_load),
        .o_BranchE(id_branch),
        .o_JalE(id_jal),
        .o_JalrE(id_jalr),
        .o_MemSrcE(id_memSrc),
        .o_ResultSrcE(id_resultSrc),
        .o_AmoFunct5E(id_amo_op),
        .a1(is_Rs1E),
        .a2(is_Rs2E),
        .is_Rs1(is_Rs1),
        .is_Rs2(is_Rs2)
    );

```

// EX Stage - Execute

```

EX ex_module (
    .clk(clk),
    .rst_n(rst_n),
    .i_rs1_data(id_rs1_data),
    .i_rs2_data(id_rs2_data),
    .i_Pc(id_pc),
    .i_AlucCtrl(id_alu_ctrl),
    .i_Func3(id_func3),
    .i_Rd(id_rd),
    .i_RegSrc(id_regSrc),
    .i_Imm(id_imm_data),
    .i_Sel1(id_sel1),
    .i_Sel2(id_sel2),
    .i_Load(id_load),
    .i_Branch(id_branch),
    .i_Jal(id_jal),
    .i_Jalr(id_jalr),
    .i_MemSrc(id_memSrc),
    .i_ResultSrc(id_resultSrc),
    .i_amo_funct5(id_amo_op),
    .i_is_amo(ex_amo_op),

```

```

.o_DataStoreM(ex_data_store),
.o_RdM(ex_rd),
.o_ResultM(ex_result),
.o_Func3M(ex_func3),
.o_RegSrcM(ex_regsrc),
.o_MemSrcM(ex_memsrc),
.o_ResultSrcM(ex_resultsrc),
.o_PcM(ex_pc),
.o_LoadM(ex_load),
.o_Result(result_to_if),
.o_Boj(ex_boj),
.o_Jalr(ex_jalr),
.o_Imm_data(ex_imm_data),
.fwdwire1(fwdwire1),
.fwdwire2(fwdwire2),
.i_mux1_sel(mux1_sel),
.i_mux2_sel(mux2_sel),
.i_ex_result(ex_result),
.i_w_data(w_data)
);

```

// MEM Stage - Memory Access

```

MEM mem_module (
    .clk(clk),
    .rst_n(rst_n),
    .i_Result(ex_result),
    .i_DataStore(ex_data_store),
    .i_Pc(ex_pc),
    .i_Func3(ex_func3),
    .i_Load(ex_load),
    .i_MemSrc(ex_memsrc),
    .i_ResultSrc(ex_resultsrc),
    .i_RegSrc(ex_regsrc),
    .i_Rd(ex_rd),
    .i_is_amo(ex_amo_op),
    .i_amo_funct5(m_amo_op),
    .o_RdW(m_rd),
    .o_RegSrcW(m_regSrc),
    .o_ResultSrcW(m_resultsrc),
    .o_ResultW(m_result),
    .o_WbDataW(m_data),
    .o_Pc4W(m_pc4),
    .i_rd_ack(i_d_ack),
    .i_read_data(i_d_data),
    .o_stb(o_d_stb),
    .o_MemSrc(o_d_wr_en),

```

```

        .o_addr(o_d_addr),
        .o_wr_data(o_d_write_data)
    );

// WB Stage - Write Back
WB wb_module (
    .clk(clk),
    .rst_n(rst_n),
    .i_RegSrc(m_regSrc),
    .i_Rd(m_rd),
    .i_ResultSrc(m_resultsrc),
    .i_Result(m_result),
    .i_Wb_Data(m_data),
    .i_Pc_4(m_pc4),
    .o_RegSrc(w_regsrc),
    .o_Rd(w_rd),
    .o_Wb_data(w_data)
);

// Hazard Unit - Data Forwarding and Hazard Detection
hazard_unit fwd (
    .a1(is_Rs1E),
    .a2(is_Rs2E),
    .is_Rs1(is_Rs1),
    .is_Rs2(is_Rs2),
    .id_rd(id_rd),
    .ex_regsrc(ex_regsrc),
    .m_regsrc(m_regSrc),
    .ex_rd(ex_rd),
    .mem_rd(m_rd),
    .load(id_load),
    .load_use_hazard(o_load_use_hazard),
    .mux1_sel(mux1_sel),
    .mux2_sel(mux2_sel)
);

endmodule

```